不友好的字体绘制

`fillText(text, left, top[, maxWidth])`

绘制字体。

`strokeText(text, left, top[, maxWidth])`

字体轮廓描边。

文本绘制是非常常见的需求了，但是 `canvas` 给文本绘制提供的方法和配置却不是那么友好，当你第一次使用时，你可能会遇到以下问题：

1） `font` 问题频出

“字体样式的属性，使用和 CSS font 规范相同的字符串值。” —— MDN

https://codepen.io/huanjinliu/embed/oNQpzam?editors=1010

可以看到当配置一致时，确实是与文档CSS样式设置后的效果一致的。

上面示例中我们配置了：`bold italic small-caps 18px/2 serif`，分别对应着：

`[font-weight] [font-style] [font-variant] [font-size]/[line-height] [font-family]`

这看起来没有问题，那又为何说它问题频出？正是因为它的简写形式。

① 上面的简写配置有着各种限制，只要其一不符合都会直接导致因解析失败。

如果 `font` 字体相关的属性的简写：

- 必须包含以下值：
  - `<font-size>`
  - `<font-family>`
- 可以选择性包含以下值：
  - `<font-style>`
  - `<font-variant>`
  - `<font-weight>`
  - `<line-height>`
- `font-style`，`font-variant` 和 `font-weight` 必须在 `font-size` 之前
- 在 CSS 2.1 中 `font-variant` 只可以是 `normal` 和 `small-caps`
- `line-height` 必须跟在 `font-size` 后面，由 "/" 分隔，例如 "`16px/3`"
- `font-family` 必须最后指定

上面最需要注意的是以下两点：

1. `font-size`、`font-family` 是必须有的。
2. `font-family` 必须在最后指定。

② `font` 解析失败不会有任何警告或报错，解析失败就直接使用默认设置：10px sans-serif。

当没有警告或报错，如果一开始你并不知道字体的效果，你就没办法确保字体效果的准确性。

提供一个最佳实践 —— 字体配置加上引号

`font` 的最稳妥的配置是：`${尺寸} "${字体名称}"`。

字体名称加上引号后可以规避很多因字符问题而导致解析错误的情况，如：`18px 123 font`、`18px self.font` 等等，

2）为什么没有绘制？

```
const canvas = document.getElementById('canvas');
const ctx = canvas.getContext('2d');

ctx.fillText("Hello world", /* offsetX / 0, / offsetY */ 0);

console.log(ctx.font);
```

当你第一次使用 `canvas` 绘制文本时，你大几率会直接设置文本的偏移为 `(0, 0)`，然后就会掉入一个陷阱：什么没有渲染。

实际上，这里的第三个参数 `offsetY` 并不应该理解成文本对于画布左上角的纵向偏移，而是应该理解成：该文本基线对于画布左上角的的Y轴偏移。

什么是基线?

为了保证字母的书写整齐，它们的书写会整齐地位于一条看不见的线上，也就是我们所说的基线，不同的字体的基线位置是不同的。

ideographic —— 表意字基线。Baseline 是西方的概念，中文本身是在田字格内书写，是没有基线的说法的，但为了便于中英混排，就产生了这么一条基线用做"参考"，但是要注意的是：图中画得很贴合，实际上并不是。

在 `canvas` 中字体的默认基线位置是 `alphabetic(标准字母基线)` 也就是上图所展示的红色虚线位置。

为什么没有效果?

因为 `fillText` 中的纵向偏移实际作用于文本的基线上，而不是文本的左上角，字体绘制超出了画布区域导致看不到任何效果，如上图所示，如果你想看到文本，你需要调整文本的基线（或者多些偏移），如：

（该示例将画布坐标系向Y轴移动30px，便于说明字体消失的原因）

https://codepen.io/huanjinliu/embed/mdQpzNm?editors=1111

3）指定字体不生效

`canvas` 指定字体绘制文本，是需要等待字体加载完成才能绘制出正确的效果。

```
@font-face {
  font-weight: normal;
  font-style: normal;
  font-family: "Battlestar";
  src: url('https://storage.sunzi.cool/font/cb277d49-86b0-4e1b-b511-76e6d9e72c26.TTF');
}
```

上面的操作相信大家都认识，但是只完成上面的操作并不能保证字体已加载，特别在网络慢或字体文件过大的情况下。我们会使用 `fontfaceobserver` 这个三方库去帮我们验证是否字体已完成加载，如下：

https://codepen.io/huanjinliu/embed/LYXOKBO?editors=1111

上面的编码方式是没有问题的，但是它需要我们同时关注 `CSS` 和 `JavaScript`，有没有办法直接使用 `JavaScript`，这里就得提到一个新的工具了：`FontFace API`。

通过 `FontFace API` 注册字体

我们可以通过 `document.fonts` 拿到文档对象中的 `Fonts` 集合（即 `FontFaceSet` 对象）。

FontFaceSet对象（MDN）

管理字体的加载以及查询字体下载状态，这里重点介绍以下两个API：

`add()`

向字体集添加一个字体

`load()`

返回解析为请求的字体的列表的 Promise

使用示例如下：

https://codepen.io/huanjinliu/embed/eYQeqee?editors=1011

值得一提的是，FontFaceSet有个专门查询字体是否加载的API：`check()`，但奈何它的兼容性实在比不过 `load()`，所以我们倾向使用 `load()` 做字体是否加载的判断。